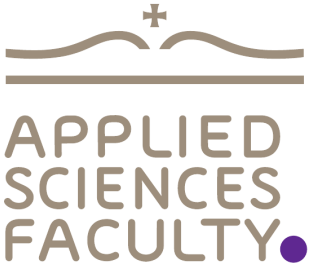
C++ — лекція 0, огляд





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

C++ це:

- Компільована,
- строго типізована,
- із статичною типізацією,
- мультипарадигмова

мова програмування загального призначення.

Вона підтримує наступні парадигми:

- Модульне програмування.
- Процедурне програмування.
- Об'єктно-орієнтоване програмування.
- Узагальнене програмування.
- Функціональне програмування.

За основу синтаксису було взято C. Підтримується висока, хоч і не абсолютна, зворотня сумісність.
Детальніше див. `http://www.cplusplus.com/info/description/`

Проста C++-програма, "Hello, world!"

```
// Це коментар. гнорується/ комплятором і.
/* Це теж
        коментар */

#include <iostream> // Директива препроцесора

using namespace std; // Включення менi з простору менi
                                        // std у глобальний простр і менi

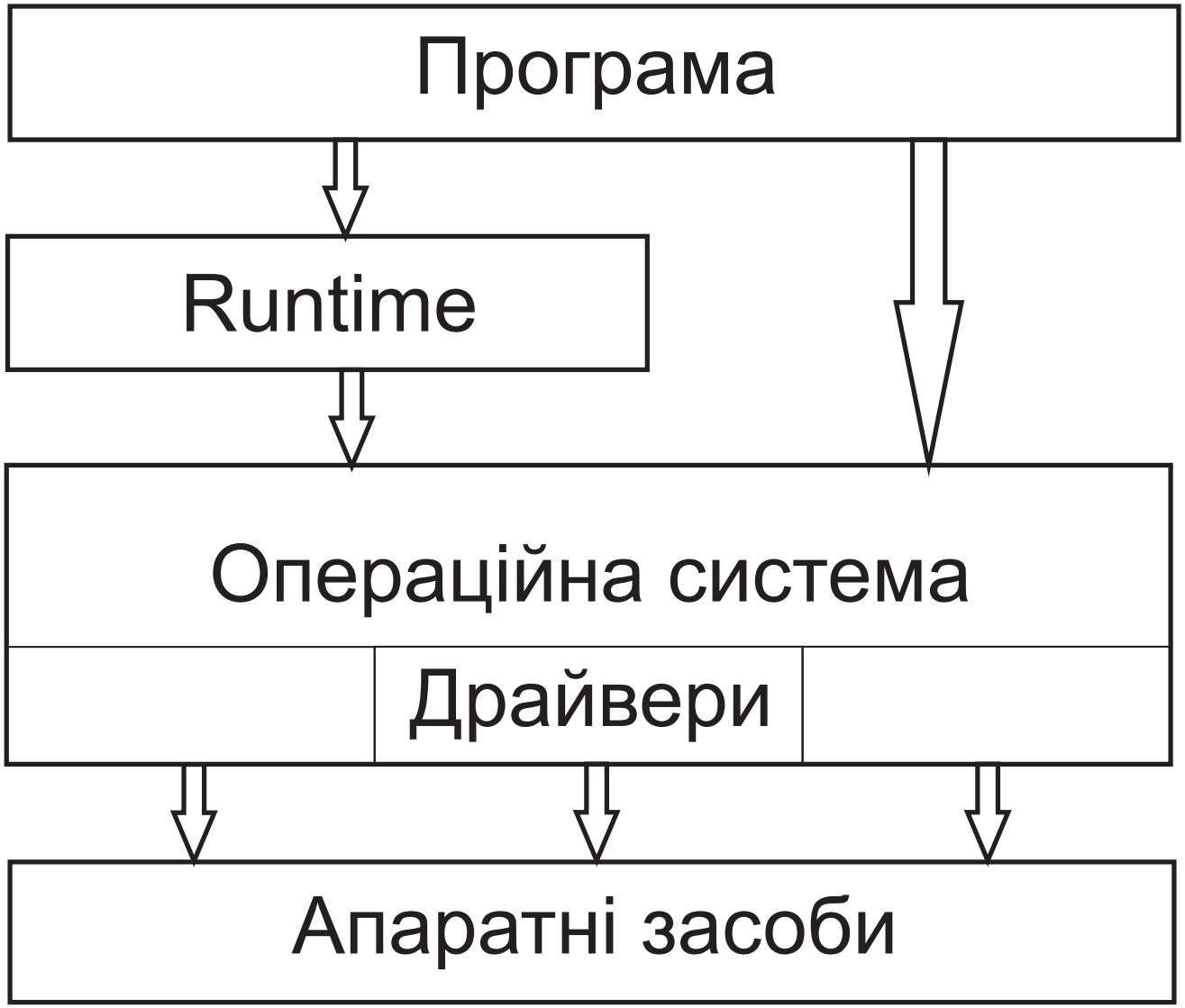
int main() // Функцяi main, точка входу в програму
{ // фгурна і дужка -- початок блоку.

    // нструкця Ii -- statement
    // Вивести текст "Hello, world!" на консоль
    cout << "Hello, world!" << endl;

    // Інструкція --- запустити зовнішню програму
    // із іменем "pause", щоб запобігти нагайному
    // закриттю програми виключно( під Windows)
    system("pause");

    return 0;    // нструкця Ii --- завершити програму з кодом 0
} // фгурна і дужка --- кнецьi блоку.
```

Код часу виконання, ”runtime”



Компілятори

Компілятор	Операційні системи	Примітки
Microsoft Visual C++	Windows	Розробка Microsoft
GCC — GNU Compiler Collection	Дуже багато[1] платформ!	Open Source. Існує практично під будь-які процесори, хоч і не завжди найкращий для них.
icc — Intel C++ Compiler	Linux, Windows, Apple Mac OS X	Розробка Intel, на кожній системі намагається бути сумісним з її засобами розробки.
Clang	Linux, FreeBSD, Windows, Apple Mac OS X, інші,	Open Source, спонсорований Apple.

Існують й інші. Зокрема вартують згадки Digital Mars C++, OpenWatcom. Однак більшість із них, з точки зору підтримки стандарту, реалізації просунутих засобів C++, підтримки та розвитку розробниками, оптимізації, тощо, поступаються перерахованим вище, якщо не по всіх перерахованих пунктах, то інтегрально. Інші ж орієнтуються на екзотичні для нас платформи.

[1] http://gcc.gnu.org/install/specific.html

Засоби, що використовуються під час програмування

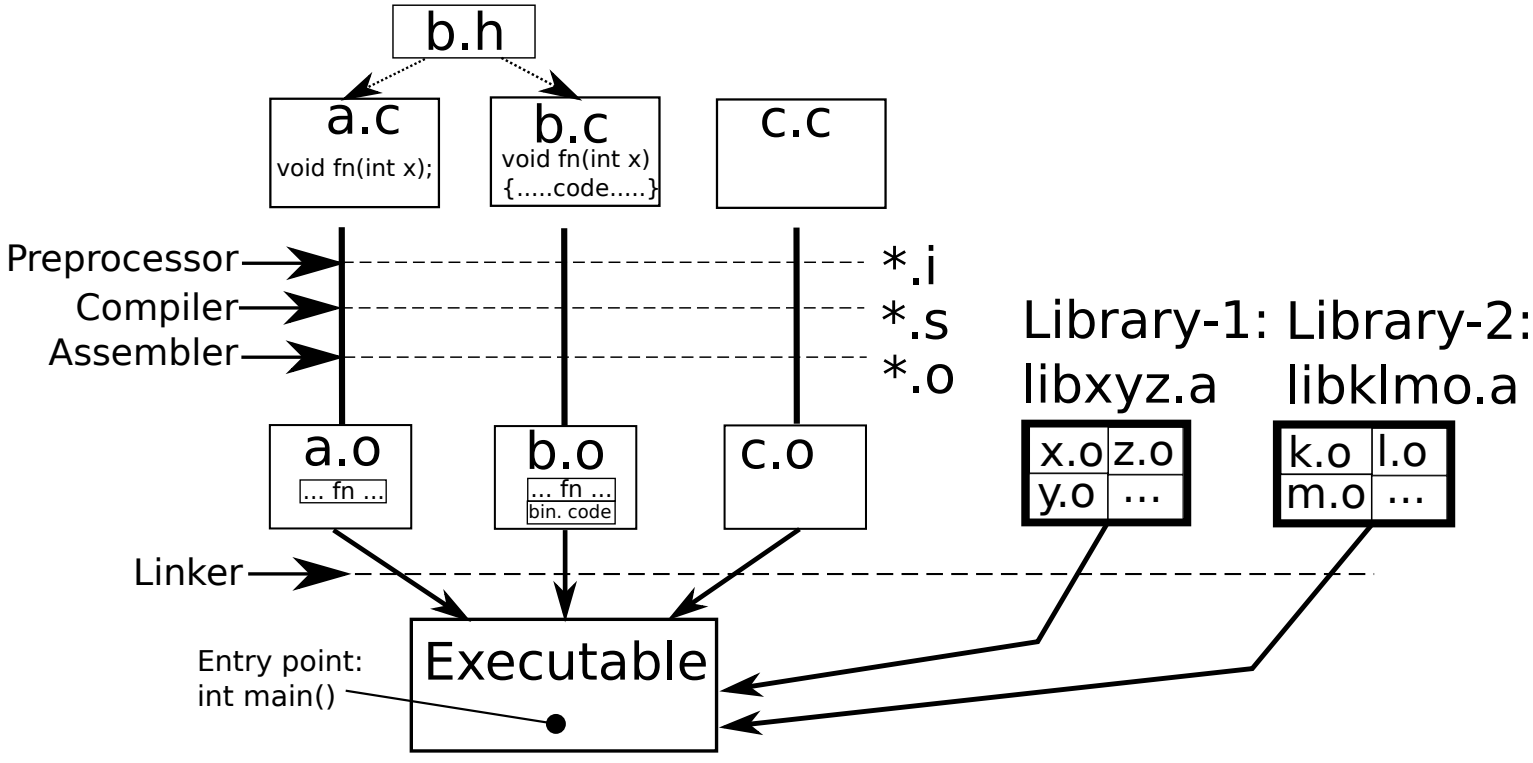
Компільовані мови:

- Компілятор.
- Компонувальник, linker.
- Зневаджувач — debugger.
- Засоби автоматичної побудови програм.
- Текстовий редактор.
- Системи контролю версій.
- Профайлер.
- Статичні аналізатори коду[2].

Середовище — Integrated Development Environment (IDE), часто включає все вище перераховане.

[2] https://habr.com/ru/company/pvs-studio/blog/673988/ – "Игра: найди ошибку в C++ коде"

Роздільна компіляція



- Детальніше про фази компіляції (трансляції) див. `http://en.cppreference.com/w/cpp/language/translation_phases`.
- Розширення – ті, що, за замовчуванням, використовує GCC.
- Проміжні файли, в нормі, не зберігаються. Щоб вони залишилися, слід скористатися ключем `-save-temps`.
- За замовчуванням, використовується AT&T стиль асемблера. Переконати використовувати стиль Інтел: `-masm=intel`. Для кращого сприйняття варто також додати `-fverbose-asm`

DeathStation 9000

- DeathStation 9000 (DS9K) – уявна дуже зла реалізація, яка старається бути максимально заважати, бути неочевидною, при тому – не порушуючи жодних стандартів.
- ◇ Приклад: реалізація strcmp для стандарту C99 може лише перевіряти стрічки на рівність – оскільки вона має сортувати за кодами літер, які не мусять бути в алфавітному порядку (ремарка – про KOI8 не чули вони...).
- Трішки подробиць: `https://web.archive.org/web/20150619140401/https://en.wikipedia.org/wiki/User:CompuHacker/CHDS9000`.
- `https://hownot2code.wordpress.com/2022/09/13/50-terrible-coding-tips-for-a-c-developer/` – як не треба робити.
- `https://hownot2code.wordpress.com/2022/09/02/4-free-code-scanning-tools-to-boost-code-quality-and-security/`
- `https://github.com/lefticus/cons_expr` – Scheme на шаблонах C++...

Про різницю C та C++

- Посилання, де описано несумісність C++ із C:
- `http://eel.is/c++draft/diff.iso` – "Annex C?(informative) Compatibility: C++ and ISO C"
- `http://eel.is/c++draft/diff.library` – "Annex C?(informative) Compatibility: C standard library"
- `https://en.wikipedia.org/wiki/Compatibility_of_C_and_C++`.
- `https://cor3ntin.github.io/posts/c/` – і трохи філософії на ту тему.

Посилання

- Список книг – 1: https://stackoverflow.com/questions/388242/the-definitive-c-book-guide-and-list.
- Список книг – 2: http://indrekis2.blogspot.com/2018/03/c.html
- Список онлайн-інструментів: http://indrekis2.blogspot.com/2020/08/blog-post_15.html.
- Способи прискорити компіляцію: https://slides.com/onqtam/faster_builds
- Що робити, щоб під ELF-системами мати dllexport/dllimport: https://gcc.gnu.org/wiki/Visibility
- Розподілена компіляція: https://github.com/distcc/distcc, https://github.com/icecc/icecream.
- Нотатки про продуктивність debug-режиму, і що взагалі значить debug-збірка: https://mropert.github.io/2019/08/03/fifty_shades_of_debug/.
- https://blog.conan.io/2019/09/02/Deterministic-builds-with-C-C++.html – "An introduction to deterministic builds with C/C++".
- https://habr.com/ru/post/470265/ – шпаргалка по абревіатурах C++, всілякі POD, SFINAE і PImpl.
- https://cor3ntin.github.io/posts/compiletime/ – огляд засобів прискорити компіляцію C++.
- https://devblogs.microsoft.com/oldnewthing/20220427-00/?p=106537 – "How do I decode a #pragma detect_mismatch error?", про виявлення деяких порушень ODR.

Посилання

- https://habr.com/ru/articles/786582/ – "Что нам недодали в C++", есей на тему.
- https://vittorioromeo.info/index/blog/debug_performance_cpp.html – "the sad state of debug performance in c++" з точки зору GameDev.
- https://www.cppstories.com/2023/storage-init-cpp/ – часи існування змінних (automatic, static, thread, dynamic) та види зв'язування (external linkage, module linkage, internal linkage, no linkage, language linkage).
- https://lucisqr.substack.com/p/why-i-think-c-is-still-a-very-attractive – "Why I think C++ is still a desirable coding platform compared to Rust"
- https://habr.com/ru/articles/786096/ – "Малоизвестные и интересные особенности C и C++", про довільні аргументи функції C, тип символів в C, нюанси виключень в деструкторах, списки виключень, ODR, нюанси шаблонів – байки, в цілому, відомі фахівцям, але цікаві.
- https://gaultier.github.io/blog/you_inherited_a_legacy_cpp_codebase_now_what.html – "You've just inherited a legacy C++ codebase, now what?".
- https://www.sandordargo.com/blog/2024/03/13/3-key-philosophies-behind-cpp – "Three key philosophies behind C++".
- https://hackingcpp.com/ – цікаві нотатки, зокрема шпаргалки.

Види проблем в коді C++ – 1

- **Well-formed** – програма відповідає синтаксичним і семантичним вимогам мови.
- **Ill-formed** – є синтаксична чи семантична помилка, яку може виявити компілятор.
  - **Ill-formed, no diagnostic required** – програма некоректна, однак діагностика дуже ускладнить компілятор, або може призвести до великого зростання часу компіляції.
    - Виконання такої програми – undefined behaviour.
    - Важливо, що компілятор **може** діагностувати такі проблеми, однак, не є зобов'язаним.
- **Implementation-defined behaviour** – стандарт не вказує, яка має бути поведінка коду в певній ситуації, однак, реалізація зобов'язана вибрати певну поведінку та документувати її.
  - Приклад такої документації для GCC: наприклад, якщо відбувається звертання до неактивного поля union, відповідні байти активного об'єкта використовуються як представлення.
- **Unspecified behaviour** – повністю на розсуд реалізації, без потреби документувати або бути узгодженим у різні моменти.
- **Undefined behaviour** (UB) – найбільш жахливий вид помилки, програма повністю некоректна, може ламати поведінку як зверху, так і знизу від місця помилки. UB буде обговорюватися окремо.
- "C++26: erroneous behaviour" – есей на тему.

Види проблем в коді C++ – 2

- **Erroneous behaviour** (C++26) – ситуація, коли реалізація може не діагностувати помилку, але сама помилка добре зрозуміла, і компілятор (можливо, ціною перевірок часу виконання) може її виявляти.
  - ◇ При цьому, ситуація залишається помилковою, однак, компілятор може вибрати конкретну поведінку, ймовірно – керовану атрибутами чи опціями, щоб програма не набувала UB.
  - ◇ Приклад – читання неініціалізованої змінної. В такій парадигмі, це все ще помилка, але компілятор може повідомляти про неї, а може допускати вважати прочитане значення коректним.
  - ◇ Такий вид запропоновано для C++26.
  - ◇ Детальніше див. відповідний документ: "Erroneous behaviour for uninitialized reads".
- **Профілі** – інша важлива пропозиція C++26, яку вирішено винести в окремий TR.
- C++26 активно працює над зменшенням кількості UB (наприклад, заборону "подорожей в часі", коли UB ламає минуле, див. також std::observable()).
- Додав (нарешті) такий засіб як hardened standard library, (покладається, поміж іншого, на контракти) – бібліотеку з більшою кількістю обов'язкових перевірок коректності: контроль виходів за межі контейнерів та адаптерів тощо. Для застосування часто достатньо просто перекомпілювати код.
- Частина роботи над C++29: каталог всіх UB, "P3596R3: Undefined Behavior and IFNDR Annexes".

Види проблем в коді C++ – посилання

- "Undefined Behavior Consequences Contest Winners"
- "Undefined behavior in C and C++", 2024.
- "Index of implementation-defined behavior"
- "The difference between undefined behavior and ill-formed C++ programs"
- "C++ needs undefined behavior, but maybe less" – чому потрібне undefined behavior, і як можна зробити, щоб його менше було, з допомогою ідеї erroneous behavior.
- "Undefined behavior can result in time travel (among other things, but time travel is the funkiest)" та "On the limits of time travel in the face of undefined behavior in C".
- "Defining All Undefined Behavior and Leveraging Compiler Transformation APIs"
- "What You Need to Know when Optimizations Changes the Behavior of Your C++". "Samples note: Use comments to describe what code does, not what you wish the code would do".
- "KLEE Symbolic Execution Engine" – експериментальний проект математичної перевірки коректності програм.
- "Uninitialized Variables", 2011 – стаття вже трохи постаріла, але ще актуальна.
- "What Every C Programmer Should Know About Undefined Behavior #1/3", "#2/3", "#3/3", 2011 – теж старіша серія статей – дещо з того часу покращилося, але актуальна, від розробників CLang.

Безпека C++ та coding guidelines

- "C++ safety, in context", 2024 – огляд проблеми і способи вирішення.
  - ◇ "P2795R5 "Erroneous behavior for uninitialized reads" – спроба у C++26 позбутися частини проблем з неініціалізованими змінними, не погіршивши ситуацію з помилками взагалі. Для цього, на додачу до undefined і unspecified behavior запропонували erroneous behaviour – "точно помилка, але поведінка визначена".
- "High Integrity C++ (HIC++)" – на момент написання цього тексту, версія 4.0, 2013 року.
- "Google C++ Style Guide", 2024.
- "MISRA C++:2023", охоплює C++17.
- "Joint strike fighter air vehicle C++ coding standards", 2005.
- "C++ Core Guidelines" – спроба покращити C++ від її творців, 2024.
- "Could Herb Sutter's call to action for C++ safety be acheived soon?" – про засоби компілятора, від попереджень до санітайзерів, для пошуку проблем в коді.
- "Crate-training Tiamat, un-calling Cthulhu:Taming the UB monsters in C++" – великий есей від Герба Саттера про боротьбу з UB в C++
- "How to Secure Existing C and C++ Software without Memory Safety" – про захист стеку (як тільки на змінну в стеку береться вказівник, компілятор ту змінну розташовує в окремій ділянці TLS), викликів функцій (перевірки відповідності типів функцій додаються компілятором, Control-Flow Integrity), купи (розмежування різних даних в різних купах) і вказівників (різноманітні рандомізації і адресного простору і вказівників). Дозволяє дуже ускладнити експлуатацію вразливостей, навіть коли безпеки пам'яті немає.

Огляди нових засобів стандартів

- `https://oleksandrkvl.github.io/2021/04/02/cpp-20-overview.html` – "All C++20 core language features with examples", квітень 2021.
- `https://www.cppstories.com/2022/20-smaller-cpp20-features/` – "20 Smaller yet Handy C++20 Features", січень 2022.
- `https://mariusbancila.ro/blog/2022/12/23/the-cpp23-standard-break-down/`
- ◇ `https://mariusbancila.ro/blog/2022/01/17/three-cpp23-features-for-common-use/`
- `https://github.com/AnthonyCalandra/modern-cpp-features` – C++11 – C++20.
- `https://github.com/steve-downey/papers/blob/master/wg21-status.org` – "C++ 23 Status Report".
- "8 More C++23 Examples" – std::ranges::to<>; std::print; std::optional Monadic Operations; std::generator; std::mdspan; cartesian_product, enumerate, and zip; chunk, slide and stride; chunk, enumerate and fold_left;
- Шпаргалка "C++23 Language Features and Reference Cards ".
- ◇ Шпаргалка "C++23 Library Features and Reference Cards".
- "C++26: Deprecating or removing library features".
- "C++26: more constexpr in the standard library".
- "C2y: Hitting the Ground Running", 2025 – countof для масивів, окрема керувальна послідовність для вісімкових чисел, інтервали чисел в case, мітки для break і contiune, бітові операції типу циклічних зсувів.

Бібліотеки

- Кілька великих списків:
  - ◇ Список від cppreference
  - ◇ awesome-cpp
  - ◇ awesome-hpp
- Проект реалізацій пропозицій до стандартної бібліотеки: `https://www.bemanproject.org/`.
- "Abseil - C++ Common Libraries" – доповнення до стандартної бібліотеки від Google.

Інше

- "Looking for Missed Alarm Bugs in a Formal Verification Tool" – про Alive2, інструмент перевірки коректності коду після оптимізації, та YARPGen – генератор рандомних програм без UB.
- "the-good-and-bad-of-cpp-as-a-rust-dev"
- "Logging all C++ destructors, poor mans run-time tracing" – про можливість логування викликів всіх функцій, засобами компіляторів (GCC/CLang), на прикладі деструкторів.
- "Vector math library codegen in Debug" – есей на тему продуктивності Debug-компіляцій.
- "Cosmopolitan Third Edition" – реалізація LibC, яка дозволяє запускати той же бінарник на Linux, MacOS, Windows, FreeBSD, OpenBSD, NetBSD – безумна штука!
- "Lesser known tricks, quirks and features of C" – дуже цікавий список фокусів, різного рівня шкідливості.
- Книга "C++ Brain Teasers: Exercise Your Mind", Anders Schau Knatten, 2024.
- "Learning to read C++ compiler errors: Failing to create a shared_ptr" – есей на болючу тему.
- `https://cppquiz.org` – **потужний опитувальник по C++.**

- Серія статей з прикладами оптимізацій, від одного з розробників LLVM:
  - ◇ "Compiler Optimization in a Language you Can Understand"
  - ◇ "Compiler Optimization in a Language you Can Understand – The Appendix"
  - ◇ "Introduction to Scalar Evolution with Iteration-based Intuition"
  - ◇ "Loop to Constant Computation"
- "Common Misconceptions about Compilers"
- "How to Create a Custom C++ Compiler Extension".
- "Modern source-to-source transformation with Clang and libTooling" – як писати утиліти для автоматичного рефактирингу, стаття 2014 року, тому подробиці застаріли, однак ідея і підходи актуальні.
- "C++ Is An Absolute Blast" – надміру екзальтована стаття, з більшістю конкретних пунктів якої я не згоден, але згоден з духом.

- Серія статей про прогрес в декомпіляції: "30 Years of Decompilation and the Unsolved Structuring Problem: Part 1", "Part 2" та "Decompiling 2024: A Year of Resurgence in Decompilation Research".
- "What was the strangest coding standard rule that you were forced to follow?"
- "What is the best comment in source code you have ever encountered?"
- "C++ Memory Management: Write leaner and safer C++ code using proven memory-management techniques", 2025.

- So are those “zero-cost abstractions” in the room with us right now?



